

مراحل پیشنهادی برای پیاده سازی طرح پروپوزال – فاز اول
در ابتدا جدول مختصری درباره تفاوت RAG متنی با رابطه ای

ویژگی	RAG متنی	RAG رابطه ای (SQL-RAG)
نوع داده	متن بدون ساختار (documents)	داده های ساختاری (جدول، ستون ها، روابط)
هدف خروجی	پاسخ زبانی	کوئری SQL صحیح
نوع embedding	فقط متن (پرسش و اسناد)	متن + ساختار پایگاه داده
واحد بازیابی	پاراگراف یا سند	جدول، ستون، یا رابطه
نوع context	متن بازیابی شده	توصیف schema + metadata
ارزیابی	دقت پاسخ با معیار BLEU و ROUGE	دقت کوئری (Execution Accuracy)
تعامل با کاربر	فقط برای پاسخ نهایی	برای اصلاح کوئری و بازخورد آموزشی

در این حالت، هدف تولید پاسخ نیست، بلکه تولید کوئری SQL صحیح برای داده های ساختاری است.

مرحله ۱: دریافت پرسش زبان طبیعی از کاربر

مثال: کدام مشتریان در ماه گذشته بیش از ۵ میلیون تومان خرید کرده اند؟

مرحله ۲: تولید embedding برای پرسش و ساختار داده رابطه ای

در داده های رابطه ای، علاوه بر embedding پرسش (مثل RAG متنی)، باید embedding ساختار داده (schema) را هم داشته باشیم:

پرسش کاربر ← embedding زبانی

جدول و ستون ها ← embedding ساختاری شامل نام، نوع داده، و رابطه

به طور مثال:

موجودیت	متن توصیفی برای embedding
جدول Customers	Table Customers: اطلاعات مشتریان شامل, CustomerID, Name, JoinDate
ستون PurchaseAmount	Column PurchaseAmount: مقدار خرید مشتری

این توصیف‌ها با همان مدل embedding به بردار تبدیل می‌شوند و در پایگاه داده برداری ذخیره می‌شوند تا براساس شباهت پرسش کاربر با این توصیف‌ها، واحد‌های بازیابی مناسبی انتخاب شوند.

مرحله ۳: تطبیق (Schema Linking)

در این مرحله، شباهت معنایی بین پرسش و عناصر schema محاسبه می‌شود: کدام جدول‌ها و ستون‌ها بیشترین ارتباط معنایی با پرسش دارند؟ مثلاً:

- مشتریان ← جدول Customers
 - خرید کرده‌اند ← ستون PurchaseAmount
 - ماه گذشته ← ستون PurchaseDate
- به این ترتیب، سیستم context ساختاری را بازیابی می‌کند.

مرحله ۴: غنی‌سازی ورودی مدل (Context Enrichment)

حالا LLM باید بداند که چه طرحواره‌ای در دسترس است. در ورودی مدل می‌نویسیم مثلاً:

```
1 User Question:
2 کدام مشتریان در ماه گذشته بیش از ۵ میلیون تومان خرید کرده‌اند؟
3
4 Context (Database Schema):
5 Table Customers(CustomerID, Name, JoinDate)
6 Table Purchases(PurchaseID, CustomerID, PurchaseAmount, PurchaseDate)
7 Relation: Purchases.CustomerID = Customers.CustomerID
```

مرحله ۵: تولید کوئری SQL

در این گام، مدل زبانی کوئری را تولید می‌کند به طور مثال:

```
1 SELECT c.Name
2 FROM Customers c
3 JOIN Purchases p ON c.CustomerID = p.CustomerID
4 WHERE p.PurchaseAmount > 5000000
5 AND MONTH(p.PurchaseDate) = MONTH(CURRENT_DATE - INTERVAL 1 MONTH);
```

مرحله ۶: اعتبارسنجی و بازخورد (Feedback & Verification)

در بسیاری از سیستم‌های پیشرفته، خروجی SQL قبل از اجرا بررسی می‌شود:

- آیا ستون‌ها وجود دارند؟
- آیا نوع داده صحیح است؟
- آیا نتایج غیرمنتظره است؟

اگر پاسخ درست نبود، سیستم با استفاده از بازخورد کاربر یا پیام خطای SQL فرآیند اصلاح (Feedback Loop) را انجام می‌دهد و کوئری جدیدی تولید می‌کند.

*- نکات کلیدی که نیازمند تنظیم است:

- ۱- توصیف embedding را برای داده‌های ساختاری بازطراحی کنی تا بیشترین دقت را داشته باشد،
- ۲- مکانیزم بازیابی را برای schema linking تنظیم کنی،
- ۳- و در مرحله generation، مدل زبانی را به تولید SQL هدایت کنی، نه پاسخ متنی.

هدف از طراحی متن embedding برای هر موجودیت:

- هر سطر از جدول embedding مثل «Table» یا «Column» یک واحد بازیابی (retrieval unit) است.
- پرسش کاربر با این واحدها مقایسه می‌شود. پس لازم است توصیف embedding طوری نوشته شود که:
- ۱- از نظر معنایی نزدیک‌ترین موجودیت به پرسش انتخاب شود یعنی Schema Linking دقیق
 - ۲- مدل زبانی هنگام تولید SQL بفهمد موجودیت‌ها دقیقاً چیست و چطور استفاده می‌شود.

ساختار پیشنهادی متن embedding برای هر نوع موجودیت

۱. برای Table (جدول)

```
<این جدول مرتبط با <دامنه داده>. <ColumnList> توضیح معنایی کاربرد جدول <شامل ستون‌های>: 1 Table <TableName>
```

نمونه:

Table Customers: اطلاعات مشتریان شامل CustomerID, Name, JoinDate است. این جدول برای نگهداری مشخصات اشخاصی است که خرید انجام داده‌اند.

♦ مؤلفه‌های پیشنهادی:

بخش	توضیح	چرا لازم است
نام جدول (TableName)	مثلاً Customers	برای حفظ شفافیت و اتصال SQL
توضیح معنایی کوتاه	مثلاً «اطلاعات مشتریان»	کمک به مدل برای درک دامنه
لیست ستون‌ها (ColumnList)	فقط ۳-۵ ستون کلیدی	به retrieval کمک می‌کند بفهمد چه اطلاعاتی دارد
دامنه داده یا نقش	مثلاً «نگهداری اطلاعات اشخاص»	درک معنایی بهتر برای کوئری‌های ترکیبی

۲. برای Column (ستون) :

```
1 Column <ColumnName> in <TableName>: <DataType> توضیح معنایی مقدار این ستون، نوع داده.
```

نمونه:

Column PurchaseAmount in Purchases: مقدار پول پرداختی هر خرید، از نوع عددی (float)، برای محاسبه مجموع و میانگین خریدها استفاده می‌شود.

۳. برای Relation (کلید خارجی یا ارتباط بین جداول):

```
1 Relation: <Table1.ColumnX> ↔ <Table2.ColumnY>, توضیح رابطه.
```

نمونه:

Relation: Customers.CustomerID ↔ Purchases.CustomerID، این رابطه هر خرید را به مشتری مربوطه متصل می‌کند.

چرا مهم است؟

- کمک به مدل برای تشخیص JOIN های درست
- کاهش خطا در زنجیره کوئری‌های چندجدولی

نکات کلیدی برای طراحی توصیف embedding ها

نکته	توضیح
از واژه‌های عمومی و نزدیک به زبان کاربر استفاده کن	مثل «مشتری»، «خرید»، «تاریخ عضویت»، نه نام‌های فنی صرف (Cust_ID)
تعداد کلمات هر embedding حدود ۳۰ تا ۶۰ توکن باشد	تا نه بیش از حد کوتاه (فقدان اطلاعات) و نه بیش از حد طولانی (کاهش تمایز) شود
از توصیف نقش کاربردی استفاده کن	مثل «برای محاسبه»، «برای نگهداری»، «برای فیلتر کردن»
بین زبان طبیعی و فنی تعادل برقرار کن	مثلاً «Table Orders» (سفارش‌ها): نگهداری اطلاعات سفارش و پرداخت‌ها
از اطلاعات نوع داده (numeric, text, date) استفاده کن	در پرسش‌های محاسباتی کمک زیادی می‌کند
از مقادیر نمونه واقعی استفاده نکن	چون در retrieval برداری نویز ایجاد می‌کند

نتیجه نهایی

برای دستیابی به بهترین نتیجه در بازایی schema:

- برای Table: توصیف مفهومی + نام ستون‌ها + نقش داده
- برای Column: نام + نوع داده + معنای مقدار + کاربرد
- برای Relations: دو ستون درگیر + نوع ارتباط (one-to-many) و معنای آن

هدف بعدی:

تبدیل خودکار Schema پایگاه داده به مجموعه‌ای از متن‌های embedding-ready به صورت زیر:

متن توصیفی برای embedding	نوع	موجودیت
Table Customers: شامل اطلاعات مشتریان با ستون‌های CustomerID, Name, JoinDate	Table	Customers
Column PurchaseAmount in Purchases: مقدار خرید مشتری، نوع داده numeric، برای محاسبه مجموع خریدها.	Column	PurchaseAmount